

Exercise 1 — Build it from a prompt

A self-paced companion to the classroom version of Exercise 1

Stand-alone version of Exercise 1, for working on your own. The classroom version (see `README.md`) saves its punchline for a live discussion; this version explains the point up front, since nobody is in the room to reveal it for you.

What you are building

A small piece of financial software: a way for an investor to sell shares of stock they hold, and see the money and profit that sale produced. Nothing more — the point is what happens when you build *even something this small* from requirements that only look complete.

The requirements

Notes from a conversation with the product owner. This is everything you get.

Our investors hold stock in their portfolios, and they need to be able to sell it.

When an investor sells, tell them how much money the sale brought in and what profit they made on it. The profit matters — they have to report it to the tax authority, so it needs to be right.

There is one complication. We buy the same stock more than once, at different times and at different prices. So the cost of the shares being sold depends on which shares you consider sold.

The money from a sale goes into the investor's cash balance.

Your test investor is **Alice**. She bought 10 shares of AAPL at 100.00, and later another 5 shares at 120.00. Today's market price for AAPL is 150.00.

Nowhere does it say **which** shares count as sold first — and that choice changes the profit number that ends up on somebody's tax return. That is not a mistake in this document: real requirements are incomplete like this all the time.

Pick a language

Java and Python are the two most common choices, but use whichever language you're comfortable with — the exercise doesn't depend on it. Pick one, then use the matching block below.

Java

- Java 21, Maven, a standalone project with its own `pom.xml`.
- JUnit 5 for tests.
- Every monetary amount as `BigDecimal` — never `double` or `float`.

Python

- Python 3.11+, a standalone project (`pyproject.toml` or `requirements.txt`).
- `pytest` for tests.
- Every monetary amount as `decimal.Decimal` — never a plain `float`.

Another language — same idea: an idiomatic project, a real test framework, and an exact (not floating-point) representation for money.

The suggested prompt

Copy this to your AI assistant, filling in the technical block for your language. Tweak the wording if you like, but don't add information the brief above doesn't contain — and don't tell the assistant which accounting method to use; that decision is what this exercise is about.

SUGGESTED PROMPT

Build a small piece of software for selling stock from an investor's portfolio.

Investors hold stock, bought at different times and possibly at different prices. When an investor sells shares, tell them how much money the sale brought in and what profit they made. The profit needs to be exactly right, because it gets reported to a tax authority. The cost of the shares being sold depends on which shares you consider sold — the same stock may have been bought more than once, at different prices. The money from a sale goes into the investor's cash balance.

To try it out, use this example: an investor named Alice bought 10 shares of AAPL at 100.00, and later 5 more shares at 120.00. Today's market price for AAPL is 150.00.

[Paste in the technical block for your chosen language here.]

Write tests for this as you build it.

Talk to the assistant as much as you like after that. What matters is that you never write down a fuller specification, acceptance criteria, or a class diagram — anything it needs to know, you say out loud in the conversation.

Build it

Go back and forth with the assistant until:

1. the code builds;
2. the tests it wrote pass; and
3. you can sell some of Alice's shares and see a proceeds figure, a profit figure, and an updated cash balance.

Before you trust that green build

Stop here, before you move on, and answer honestly:

Which accounting method did your assistant pick — did you tell it to, or did it decide on its own?

There are at least three defensible answers to "which shares count as sold": oldest first (FIFO), blended average, or newest first (LIFO) — all real accounting methods. Your assistant picked one *silently*, since the brief never named one.

More importantly: **the assistant wrote the code from its own reading of the brief, then wrote the tests from that same reading.** The tests never checked against a business rule — there was none to check against. A test suite that only has to agree with the implementation that produced it will always pass. The same blind spot hits anywhere else the brief stayed quiet: selling zero shares, more than you hold, a negative price.

Check yourself

Once you've built something, open `conformance.md` — a checklist of concrete numbers and edge cases. Run it against your implementation. A low score isn't failure: the brief made some answers genuinely unknowable in advance.

Before you look at your score, write down:

What did I never tell the assistant, that it had to decide on its own?

What's next

This exercise handed you a brief and asked you to talk your way to working software. Exercise 2 hands you the same kind of problem again — but this time the behaviour lives in versioned specification files that exist **before** any code does, reviewable line by line instead of buried in a conversation only you saw.